Join our community, grow your skills, and connect with creators worldwide.

Sign In Join Now

Or sign in with Epic Games

Networking Overview

Learn about networking in Unreal Engine including architecture concepts and the available replication systems.

In multiplayer game sessions, game state information is communicated between multiple machines over a network connection. In certain cases, multiplayer game sessions involve dozens of game clients connected to a single server, while others may involve only a few clients connected to a server. Regardless of the number of connections, the purpose of networking gameplay is to ensure that all players experience a shared game environment. Because of some key fundamental differences between how multiplayer and single-player games handle gameplay, Unreal Engine (UE) features a broad set of tools and systems to help you develop your multiplayer game. This document provides an overview of the key concepts and terminology you need to understand when developing your multiplayer game. It also introduces the core tools and systems that Unreal Engine provides for assisting you with your multiplayer game development.

Plan Early for Multiplayer

If there is any possibility that your project might need multiplayer features at any time, you should plan for it early in the development process. Although Unreal Engine provides a wide range of tools and systems to help you develop your multiplayer game, the process of building gameplay with networking in mind will consume much more time compared to a single-player game. In the long run, your project will be easier to maintain if you have to add multiplayer features later, but it will still require a significant amount of time and effort to add multiplayer features to a single-player game.

If you do not design your project with multiplayer in mind from the beginning, replicating a codebase that you have developed for single-player gameplay will require you to modify your single-player gameplay and rewrite large sections of gameplay functionality. You also might need to relocate your single-player gameplay code to different classes or functions to ensure that it works correctly in a client-server or peer-to-peer networking environment.

Unreal Engine Networking Architecture

UE uses the **client-server architecture** for networked multiplayer games. There are two types of networked game sessions: dedicated servers and listen servers. In a dedicated server, a single server is an headless program, and game clients must connect to this server to play in a multiplayer game. In the instance of a listen server, one of the game clients acts as the server for the other clients to connect to. In this case, the client that also hosts the server is referred to as the listen server client or simply the listen server. In the context of a listen server, the other clients connected to the listen server are referred to as remote clients. The process of connecting multiplayer games is actually quite simple. The client-server control remains **Pawns** that take care of the majority of gameplay functionality, and the client-server control **Pawns** that take care of the majority of gameplay functionality also can be used in both single-player and multiplayer games. The only time when **Actors** are located, have their actions replicated, and when client-server controls are required is when a game session is being hosted on a dedicated server or listen server. This also actually happens by default.

Client
Client
Client